

When Metrics Move, Answers Follow in Seconds

An automated root-cause analysis engine that detects anomalies, decomposes their drivers, localises the responsible segments, and narrates the diagnosis, all inside ClickHouse.

Team Flux4 · ClickHouse Cloud | Langfuse | LibreChat

Ramyashree shetty
Bharateesh CA
Shanmathi Mohanraj
Spoorthy VV

An alert fires. Now what?

InMobi processes billions of ad events daily. When revenue or fill rate drops, a human must manually drill through dashboards — dimension by dimension — to find root cause.

The real question isn't "Did the metric move?"

It's "Why did it move, and exactly where?"

Answering this today takes hours or days — even though all the data is already in the warehouse.



Hours to diagnose

Analysts manually slice across 10+ dimensions: geo, device, OS, app, advertiser...



Easy to miss root causes

Blended metrics hide collapses. A 4.5% traffic surge can mask a fill-rate disaster.



Dashboard fatigue

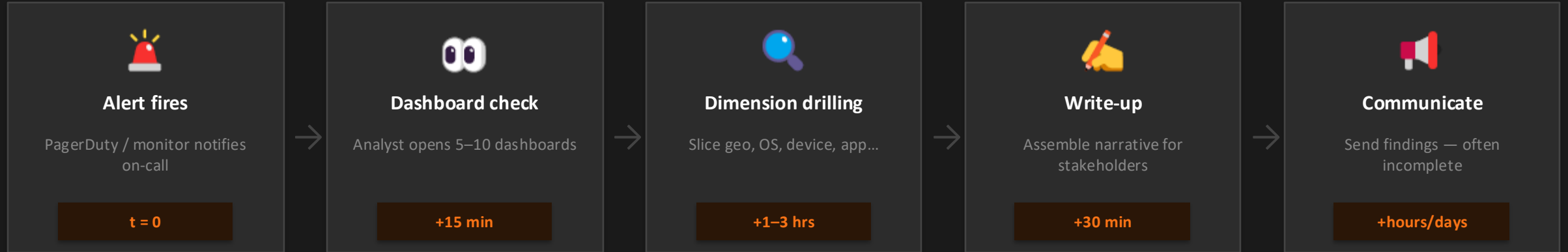
30+ dashboards, no single view of what caused this and what's been ruled out.



LLM hallucination risk

Naive AI invents numbers. A fabricated % drives wrong business decisions.

The manual investigation process today

**2–8**

hours per incident

10+

dimensions to check manually

∞

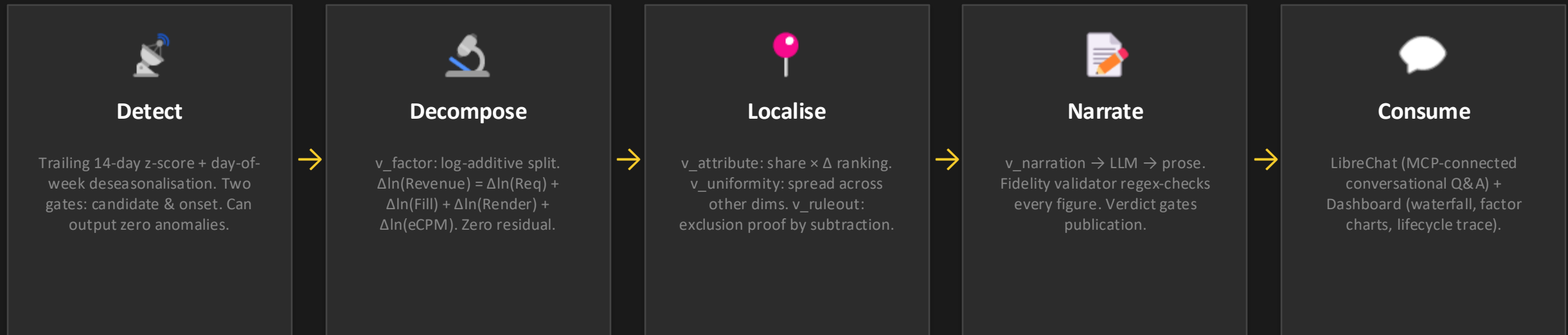
dashboards, no audit trail

0

automatic ruled-out list

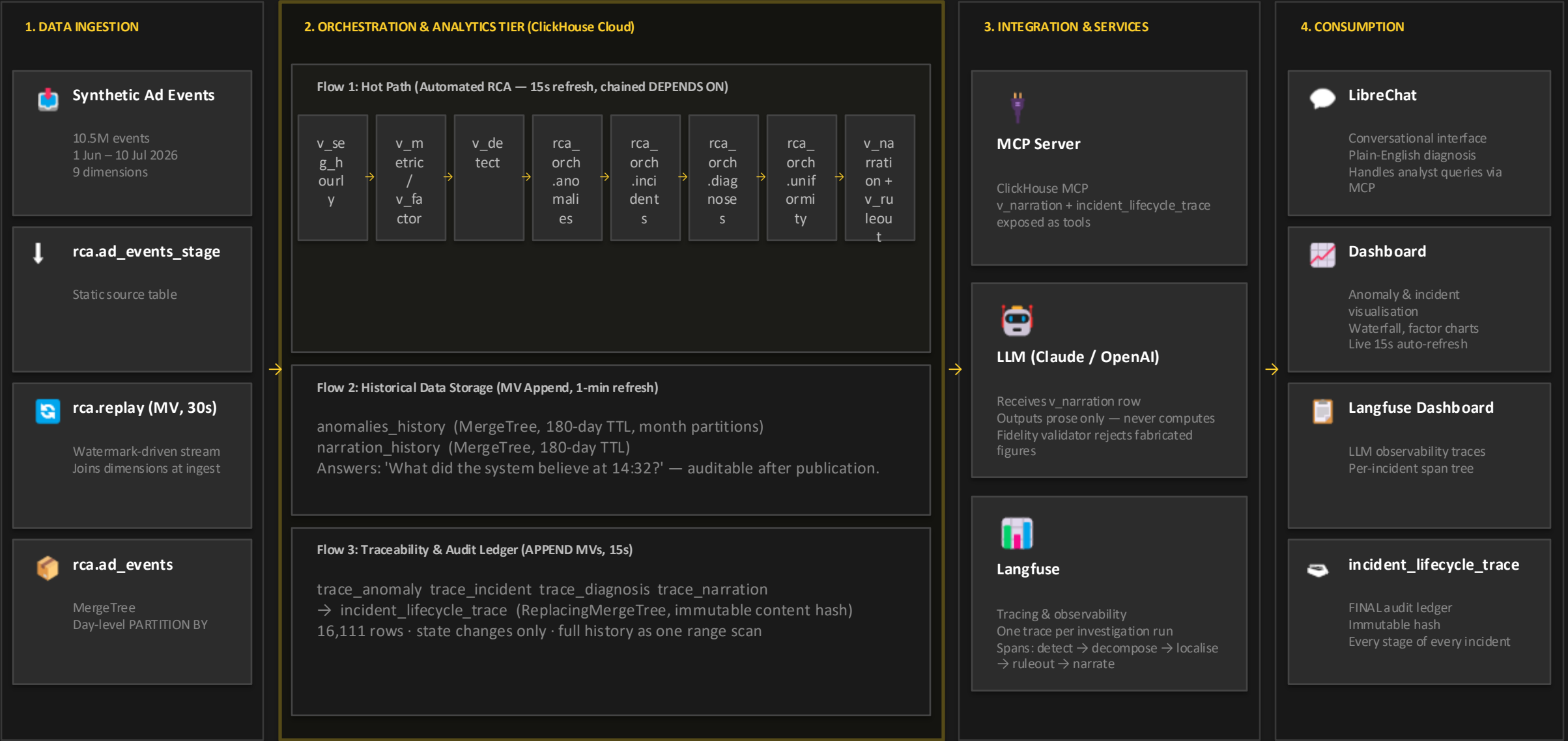
A self-explaining RCA engine

Every investigation step writes a row to a ClickHouse table. The ledger is simultaneously the trace, the ruled-out list, the attribution output, and the audit artifact — all produced by SQL, narrated by LLM.



Zero SQL written by analysts. Zero fabricated numbers. One ledger row per investigation step.

ClickHouse Native RCA Architecture: 4 Tiers, No External Orchestrator



The entire investigation is SQL

What we used	Why it mattered
11 Refreshable Materialized Views	Entire orchestration runs inside ClickHouse — no Airflow, no cron, no app logic
MergeTree / ReplacingMergeTree	MergeTree for replace-each-tick result tables; ReplacingMergeTree for uniformity & lifecycle_trace
SharedMergeTree (ClickHouse Cloud)	Cloud-native variant — no replication config needed; transparently used for all result tables
ARRAY JOIN unpivot (v_seg_hourly)	1 event → 9 (dim, val) rows in a single view; detection & attribution share one unpivot
v_metric: (num, den, is_ratio) triples	Ratios computed as sum/sum at read time — never stored, never averaged. Structurally prevents ratio-of-ratios.
v_factor: log-additive decomposition	$\Delta \ln(\text{Revenue}) = \Delta \ln(\text{Requests}) + \Delta \ln(\text{Fill}) + \Delta \ln(\text{Render}) + \Delta \ln(\text{eCPM})$ — residual zero by construction
Trailing 14-day window function	ROWS BETWEEN 14 PRECEDING AND 1 PRECEDING, partitioned by (metric, dim, val) — baseline is sum/sum over window
Day-level PARTITION BY + ORDER BY	Hot-path time-window queries land on 1–3 day partitions; ORDER BY drives segment drill-in latency
Dictionaries (HASHED layout)	dictGetOrDefault at ingest via rca.replay — no INNER JOIN, so unfilled requests keep their denominator row
ReplacingMergeTree + content hash	incident_lifecycle_trace collapses on sipHash64(stage, incident_id, record_key, details) — state changes only, not every tick

■ 11 MVs, zero external deps

The hot path fires every 15s: anomalies → incidents → diagnoses → uniformity, chained with DEPENDS ON.

■ 10.5M events, 40 days

rca.ad_events_stage (static); rca.replay streams it as a moving 30s watermark window.

■ Two detection gates

Candidate: $|z| \geq 2$ & $|\text{effect}| \geq 1\%$. Onset: $|z| \geq 4$ & $|\text{effect}| \geq 2\%$. Both must clear to open an incident.

■ ~2 KB LLM context/incident

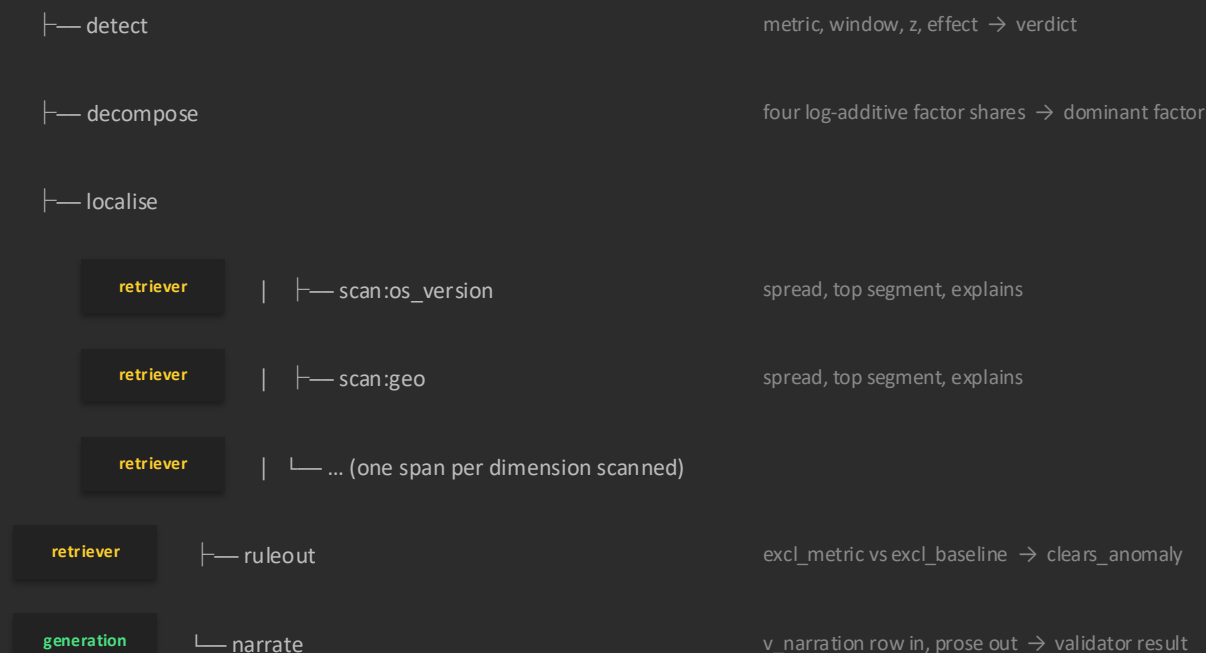
ClickHouse computes everything; v_narration emits one row. LLM writes English only — never computes.

The investigation is a trace, Langfuse captures it as one

The investigation is an ordered sequence of steps with rationale — so the Langfuse span tree and incident_lifecycle_trace are the same structure recorded twice: once for observability, once for SQL audit.

SPAN TREE — one trace per investigation run (run_id)

investigation (run_id)



root

1
.
8

trace_id is carried on rca.app_events — any figure in the diagnosis can be walked back to the span that produced it.

Why Langfuse is the right fit for RCA

Investigation = trace

Detect → decompose → localise → ruleout → narrate is already an ordered span tree. Langfuse models it natively.

Numeric fidelity audit

Every generation span carries the validator result. Failed generations are visible in the Langfuse UI, not silently swallowed.

Cost per incident

Token spend is captured per generation span — one call per incident, ~2 KB context, negligible cost.

Prompt regression detection

Any change to the narration prompt can be A/B-scored in Langfuse before touching the live pipeline.

Walk back any figure

trace_id on rca.app_events links every published number to the ClickHouse query that produced it.

Conversational RCA, powered by LLM, grounded in ClickHouse data

LibreChat is the conversational interface embedded in the dashboard. Every question the analyst asks is answered by an LLM that has been injected with the full incident attribution context — no hallucination, no generic answers.

What LibreChat enables



Incident-scoped chat

On opening an incident, LibreChat's system prompt is pre-loaded with the full attribution ledger — segment, contribution %, delta, and exclusion proof. Every answer is grounded in computed facts.



Live streaming answers

Responses stream token-by-token via an OpenAI-compatible endpoint. Analysts get instant feedback while ClickHouse re-queries run in parallel.



MCP-connected

LibreChat is wired via MCP (Model Context Protocol) to the ClickHouse backend. Follow-up questions trigger live SQL queries — the chat is a live BI interface.



No credentials in chat

API keys and ClickHouse credentials stay in server env vars. Browser localStorage holds only the session token — nothing sensitive exposed client-side.

LibreChat · Incident #3 — Android 15

Live

Why did fill rate collapse on Jun 23?

Fill rate dropped 4.51% below baseline ($z = -47.5$). The decomposer shows fill rate accounts for 100% of the revenue shortfall — not requests or eCPM. One segment explains 97.9% of the drop.

Which segment?

Android 15 devices — 9.6% of traffic. Excluding them returns global fill rate to 0.7844 vs the baseline of 0.7852, which clears the anomaly (z drops below threshold).

What was the revenue impact?

\$70.21 over three days (~4.6% of expected revenue). A 4.5% traffic increase on those days masked the collapse at the blended level — the decomposition found it anyway.

Every claim is a row. Every number is verified.

STEP	SEGMENT	DELTA	VERDICT
detector	fill_rate · global	-4.51%	ANOMALY
decomposer	factor: fill_rate	-100%	DRIVER
localiser	os_version = Android 15	97.9% contrib	CULPRIT
ruleout	excl. Android 15	0.7844 vs 0.7852	CLEAR
ruleout	os_version = iOS 18	-0.2%	NORMAL
ruleout	geo = US, IN, BR...	within ±0.3%	RULED_OUT

GENERATED NARRATION (LLM)

"Between June 23 and June 25, global fill rate fell 4.51% below the same-weekday baseline (z = -47.5). The driver is entirely fill rate: after log decomposition, fill rate accounts for 100% of the revenue factor shortfall. A single segment — Android 15 devices, 9.6% of traffic — explains 97.9% of the global drop; excluding Android 15 returns fill rate to 0.7844 versus a baseline of 0.7852, clearing the anomaly."

Revenue counterfactual: Android 15 fill collapse cost \$70.21 over 3 days (~4.6% of expected revenue) despite appearing flat at blended level — traffic surge masked the collapse.

✓ Numeric fidelity validator — all 6 figures found in ledger. Regex-checks every number. Rejects + retries on mismatch.

5 open incidents detected. Weekend decoy cleared. 0 false alarms.

5

Open incidents detected across 40 days of data

0

Weekend false alarms — Sat/Sun seasonality cleared by day-of-week factor

-63.4

Peak z-score (fill_rate Jun 23-25) — 0.237% daily CV makes 4.45% a catastrophe

2

Detection gates: candidate ($|z| \geq 2$) AND onset ($|z| \geq 4$) must both clear

Window	Metric	Days	Worst effect	Peak z	Verdict
Jun 19-21	ecpm	3	-2.56%	-18.9	OPEN
Jun 19-21	rpr	3	-2.55%	-16.3	OPEN
Jun 23-25	fill_rate	3	-4.45%	-63.4	OPEN
Jul 06-07	ecpm	2	-8.20%	-11.7	OPEN
Jul 06-09	rpr	4	-15.70%	-4.3	OPEN
Every Sat/Sun	requests	—	-19%	~0	CLEARED

Daily CV — why one threshold serves all metrics

render_rate	0.031%	most sensitive — 0.5% move = -15σ
fill_rate	0.237%	4.45% move $\rightarrow z = -63$
ctr	1.81%	noisy — 2% move is background
rpr	2.42%	high variance, onset gate needed
ecpm	2.61%	$-2.56\% \rightarrow z = -18.9$
revenue	8.20%	detrended by day-of-week factor
requests	8.50%	detrended by day-of-week factor

5 differentiating ideas that made the difference

01

Numeric Fidelity Validator

Every number in LLM-generated prose is regex-extracted and asserted against the ledger. Fabrication is impossible by construction — not by prompt engineering. Reject and retry on mismatch.

02

Decompose-First Architecture

Rate factors (fill_rate CV = 0.237%) have no weekday seasonality. v_factor takes logs so $\Delta \ln(\text{Revenue}) = \Delta \ln(\text{Requests}) + \Delta \ln(\text{Fill}) + \Delta \ln(\text{Render}) + \Delta \ln(\text{eCPM})$. Weekend seasonality never reaches the factor detectors.

03

Concentration-Based Dimension Selection

Causal dimension identified by variance in per-segment deviation across that dimension — not absolute delta. Avoids the Zipfian trap where the biggest segment always 'wins' regardless of cause.

04

Global-vs-Segmental Verdict

System can output "no segment responsible" and correctly does so for Jun 21 (every dimension fell uniformly). Competing teams returning top-N segments by delta would fabricate a false culprit.

05

Exact Log-Additive Attribution — Zero Residual

Revenue = Requests × Fill rate × Render rate × eCPM/1000. Contributions sum to exactly 100.000000% with zero residual — verified to 6 decimal places on every incident. Every factor is always computed, so the ruled-out list is generated for free.

Tools used and why each one matters

CORE ENGINE

- ClickHouse Cloud v26.5
- MergeTree / SummingMergeTree
- Materialized Views
- Dictionaries (HASHED layout)
- ARRAY JOIN unpivot
- Window functions + MAD
- chdb (embedded ClickHouse)

OSS INTEGRATIONS

- Langfuse v4 — full pipeline tracing
- LibreChat — LLM-powered chat UI
- MCP — ClickHouse ↔ LibreChat bridge

LLM / AI LAYER

- OpenRouter (LLM API)
- Numeric fidelity validator
- In-memory narration cache
- Langfuse generation traces
- Context-aware chat widget

Single threshold $z_{crit} = 4$ serves all 6 metrics without per-metric tuning — σ_{floor} derived from data. · ~2 KB LLM context per incident. · 25s view cache balances freshness with 3–8s recompute cost.

SUMMARY

From alert to answer in seconds, not hours

The Hypothesis Ledger turns ClickHouse into a self-explaining RCA engine
every diagnosis backed by SQL, every number verified.